COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S1
DATE PERFORMED:	11/08/2026	**DATE SUBMITTED:**	11/08/2026
NAME:	Ramon Brendt D. Campo	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 5.1 [QUEUE]

1. PROCEDURE OUTPUT

ILO A

```cpp
#include <iostream>
#include <queue>



void display(std::queue<char> r);
void empty(std::queue<char> r);
void size(std::queue<char> r);

int main(){
    char x = x;
    std::queue<char> q;

    //creates a queue/line
    q.push('c');
    q.push('p');
    q.push('e');
    q.push('0');
    q.push('1');
    q.push('0');


    //display
    display(q);

    //empty()
    std:: cout << q.empty() << std::endl;

    //size()
    std:: cout << "the size of the queue is " << q.size() << std:: endl;
    //front
    std:: cout <<"the front of the queue is " << q.front() << std:: endl;
    //back
    std:: cout <<"the back of the queue is " << q.back()  << std:: endl;
    //pop
    std:: cout << "removing the first element" << std:: endl;
    q.pop();
    //display
    display(q);
```

```
40      //push another element
41      q.push('R');
42      q.push('a');
43      q.push('m');
44      //display
45      display(q);
46
47     return 0;
48  }
49
50  //r is q
51  void display(std::queue<char> r){
52       //create a copy of the queue
53       std::queue<char> c = r;
54
55       //!note c.function is from stl
56       //executes it until empty
57       while(!c.empty()){
58         std::cout<<" " << c.front(); //prints and pops the front and goes to the next node
59         c.pop();
60
61       }
62
63       std::cout<<"\n";
64
65  }
66
67  void size(std::queue<char> r){
68
69     //this is the queue
70     std::queue<char> c = r;
71
72     std::cout <<"the size of the queue is: " << c.size();
73
74  }
```

Output

```
c p e 0 1 0
0
the size of the queue is 6
the front of the queue is c
the back of the queue is 0
removing the first element
p e 0 1 0
p e 0 1 0 R a m
PS C:\Users\TIPQC\Documents\DSA CAMPO>
```

ILOB

MAIN.cpp

```cpp
#include <iostream>
#include "queuell.h"


int main(){

//why front and back?

qNode<char> * front = nullptr;
qNode<char>* back = nullptr;

enqueue(&front, &back, 'R');
std::cout << front->data << " " << back->data << std::endl;
enqueue(&front, &back, 'A');
std::cout << front->data << " " << back->data << std::endl;
enqueue(&front, &back, 'M');
std::cout << front->data << " " << back->data << std::endl;
dequeue(&front, &back);
std::cout << front->data << " " << back->data << std::endl;
enqueue(&front, &back, 'R');
std::cout << front->data << " " << back->data << std::endl;

std:: cout << "Queue list \n";
display(front);
std::cout << "Front value: " << FrontVariable(front) << std::endl;

    return 0;
}

```

HEADER

```cpp
#ifndef QUEUELL_H
#define QUEUELL_H

#include <iostream>

template <typename T>
class qNode{
      public:
      T data;
      qNode* next;

};


//creating a new node

template <typename T>
qNode<T> * new_node(T newData){
    //allocates the new node
    qNode<T>* newNode = new qNode<T>;
    //sets the new data in the newNode's data
    newNode->data = newData;
    //points the next pointer of the newNode to nullptr
    newNode->next = nullptr;
    return newNode;



}

```

```
31  //===========
32  //enqueue
33  //===========
34
35
36
37
38
39  template<typename T>
40  //frontptr and backptr is defined in the int main
41  void enqueue(qNode<T>** frontPtr, qNode<T>** backPtr, T newData){
42        //create a new node
43        qNode<T>* newNode = new_node(newData);
44
45        //inserting to an empty queue
46        //checks if the queue is empty
47        if((*frontPtr) == nullptr && (*backPtr) == nullptr){
48        (*frontPtr) = newNode;
49        (*backPtr) = newNode;
50        }
51
52
53        //inserting an item into a non empty queue
54        //point the backPtr next to the newNode
55        (*backPtr)->next = newNode;
56        (*backPtr) = newNode;
57
```

```
62
63  //============
64  //dequeueing
65  //============
66
67  template <typename T>
68  void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr) {
69        //create a temporary node to store the node to be deleted
70        qNode<T>* deleteNode = nullptr;
71        deleteNode = (*frontPtr);
72
73        //check if the queue is only one 1 node
74      if((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr){
75            (*frontPtr) = nullptr;
76            (*backPtr) = nullptr;
77            delete deleteNode;
78            return;
79
80      }
81        //deleting of the node
82        (*frontPtr)= deleteNode->next;
83        delete deleteNode;
84
85
86
87  }
```

```
86
87  }
88  //================================
89  //display all elements in the list
90  //================================
91
92  template <typename T>
93  void display(qNode<T>* frontPtr){
94     if(frontPtr == nullptr){
95        std::cout<< "queue is empty \n";
96        return;
97     }
98
99     qNode<T>* current = frontPtr;
100    while (current != nullptr){
101     std::cout << current->data << "->";
102     current= current->next;
103    }
104    std::cout<< "NULL \n";
105
106 }
107
108 //============================
109 //return the front variable
110 //============================
111 template <typename T>
112 T FrontVariable(qNode<T>* frontPtr){
113
114    if(frontPtr != nullptr) {
115     return frontPtr->data;
116    }
117
118 }
119 #endif
```

Output

```
R R
R A
R M
A M
A R
Queue list
A->M->R->NULL
Front value: A
```

ILO B.2

HEADER

```
#ifndef QUEUEARR_H
#define QUEUEARR_H

#include <iostream>

// Array-based circular queue
template <typename T>
class queueArr {
    private:
        // Creates a dynamically allocated queue array (pointer array)
        T* q_array;
        size_t q_capacity;
        size_t q_size;
        int q_front;
        int q_back;

    public:
        // Prototypes of member functions:

        // Constructor
        queueArr(size_t capacity);
        bool empty();
        bool full();

        size_t size();
        void Clear();

        T Front();
        T Back();
```

```
31          void Enqueue(T newData);
32          void Dequeue();
33
34          // Destructor
35          ~queueArr();
36
37          // Copy constructor and assignment operator
38          queueArr(const queueArr& other);
39          queueArr& operator=(const queueArr& other);
40  };
41
42  // Constructor
43  template <typename T>
44  queueArr<T>::queueArr(size_t capacity) {
45      // Capacity of the array
46      q_capacity = capacity;
47
48      // Current size of the array
49      q_size = 0;
50
51      // Initial front and back index
52      q_front = 0;
53      q_back = -1;
54
55      // Newly allocated array
56      q_array = new T[q_capacity];
57      std::cout << "Queue created with capacity: " << q_capacity << std::endl;
58  }
```

```cpp
// Destructor
template <typename T>
queueArr<T>::~queueArr() {
    delete[] q_array;
    std::cout << "Queue destroyed." << std::endl;
}

// empty()
template <typename T>
bool queueArr<T>::empty() {
    return q_size == 0;
}

// full()
template <typename T>
bool queueArr<T>::full() {
    return q_size == q_capacity;
}

// Clear
template <typename T>
void queueArr<T>::Clear() {
    q_size = 0;
    q_front = 0;
    q_back = -1;
}

```

```cpp
// Return the element at the front of the queue
template <typename T>
T queueArr<T>::Front() {
    // Check and display if the queue is empty
    if (empty()) {
        std::cout << "Queue is empty. \n" << std::endl;
        return T{}; // Return default value of T
    }
    // Return the front element
    return q_array[q_front];
}

// Return the element at the back of the queue
template <typename T>
T queueArr<T>::Back() {
    // Check and display if the queue is empty
    if (empty()) {
        std::cout << "Queue is empty. \n" << std::endl;
        return T{}; // Return default value of T
    }
    // Return the back element
    return q_array[q_back];
}

// Size
template <typename T>
size_t queueArr<T>::size() {
    return q_size;
}

```

```cpp
// Enqueue
template <typename T>
void queueArr<T>::Enqueue(T newData) {
    // Check if the queue is full
    if (full()) {
        std::cout << "Queue is full." << std::endl;
        return;
    }

    // Compute for q_back to move it circularly (clue: use modulo)
    q_back = (q_back + 1) % q_capacity;

    // Assign the new value to the back of the queue array
    q_array[q_back] = newData;

    // Increment the q_size
    q_size++;
}

// Dequeue
template <typename T>
void queueArr<T>::Dequeue() {
    // Check if the queue is empty
    if (empty()) {
        std::cout << "Queue is empty." << std::endl;
        return;
    }

    // Compute for q_front to move it circularly
    q_front = (q_front + 1) % q_capacity;
```

```cpp
    // Compute for q_front to move it circularly
    q_front = (q_front + 1) % q_capacity;

    // Decrement the q_size
    q_size--;
}

#endif // QUEUEARR_H
```

MAIN

```cpp
#include <iostream>
#include "queueArr.h"

int main(){

    //create an object
    queueArr<int> q(6);
    q.Enqueue(1);
    std::cout << "q_front: " << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;

    q.Enqueue(2);
    std::cout << "q_front: " << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;

    q.Enqueue(3);
    std::cout << "q_front: " << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;

    q.Enqueue(4);
    std::cout << "q_front: " << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;

    q.Enqueue(5);
    std::cout << "q_front: " << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;

    q.Enqueue(6);
    std::cout << "q_front: " << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;


    q.Dequeue();
    std::cout <<"q_front: " << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;

    q.Dequeue();
    std::cout << "q_front: " << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;

    q.Dequeue();
    std::cout << "q_front: " << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;

    q.Dequeue();
    std::cout << "q_front: " << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;

    q.Dequeue();
    std::cout << "q_front: " << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;

    q.Dequeue();
    std::cout << "q_front: " << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;
    return 0;

}
```

Output

```
Queue created with capacity: 6
q_front: 1, q_back: 1, q_size: 1
q_front: 1, q_back: 2, q_size: 2
q_front: 1, q_back: 3, q_size: 3
q_front: 1, q_back: 4, q_size: 4
q_front: 1, q_back: 5, q_size: 5
q_front: 1, q_back: 6, q_size: 6
q_front: 2, q_back: 6, q_size: 5
q_front: 3, q_back: 6, q_size: 4
q_front: 4, q_back: 6, q_size: 3
q_front: 5, q_back: 6, q_size: 2
q_front: 6, q_back: 6, q_size: 1
q_front: Queue is empty.
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

HEADER

```cpp
#ifndef PRINTER_SYSTEM_H
#define PRINTER_SYSTEM_H

#include <iostream>
#include <string>

// Templated circular array queue using typename T
template <typename T>
class queueArr {
private:
    T* q_array;
    size_t q_capacity;
    size_t q_size;
    int q_front;
    int q_back;

public:
    //assigns the values of the private variables and dynamically allocates the memory
    queueArr(size_t capacity) : q_capacity(capacity), q_size(0), q_front(0), q_back(-1) {
        q_array = new T[q_capacity];
    }

    //deletes the queue
    ~queueArr() {
        delete[] q_array;
    }
    //checks if the queue is empty
    bool empty() const { return q_size == 0; }

    //Checks if the queue is full by comparing current size to total capacity
    bool full() const { return q_size == q_capacity; }

    //returns the size of the array(number of items)
    size_t size() const { return q_size; }

    //returns the capacity of the array(the maximum size the array can hold)
    size_t capacity() const { return q_capacity; }

    //Returns the element at the front of the queue
    T Front() const {
        if (empty()) return T{};
        return q_array[q_front];
    }

```

```cpp
    //Returns the element at the front of the queue
    T Front() const {
        if (empty()) return T{};
        return q_array[q_front];
    }

    //adds contents to the queue
    void Enqueue(T newData) {
        if (full()) return;
        //modulo is used here to make sure the back wraps around in index 0 to loop the queue
        //it also assigns the index position of the array
        q_back = (q_back + 1) % q_capacity;
        //assigns the new data depending on the position/index of the data inserted in the queue
        q_array[q_back] = newData;
        q_size++; //increments the size
    }


    void Dequeue() {
        if (empty()) return;
        // Advances q_front and uses modulo to wrap around to index 0 when reaching capacity
        q_front = (q_front + 1) % q_capacity;
        q_size--;
    }
};

// Job representation
class Job {
public:
    int id;
    std::string userName;
    int pages;

    //adds the id, name and the ammount of pages inside the queue
    Job() : id(0), userName(""), pages(0) {}
    Job(int id, std::string userName, int pages)
        : id(id), userName(userName), pages(pages) {}

    //displays the contents of the job function
    void display() const {
        std::cout << "[Job ID: " << id << " | User: " << userName << " | Pages: " << pages << "]";
    }
};
```

```cpp
// Printer class managing pending Job objects
class Printer {
private:
    //assigns jobs that are pending
    queueArr<Job> pendingJobs;

public:
    Printer(size_t capacity) : pendingJobs(capacity) {}

    //status getter functions that query the capacity and current count of the queue.
    bool isFull() const { return pendingJobs.full(); }
    bool isEmpty() const { return pendingJobs.empty(); }
    size_t getPendingCount() const { return pendingJobs.size(); }
    size_t getCapacity() const { return pendingJobs.capacity(); }

    //adds the job contents
    void addJob(const Job& job) {
        if (pendingJobs.full()) {
            std::cout << "\nError: Printer queue is full! Unable to add ";
            job.display();
            std::cout << std::endl;
            return;
        }

        pendingJobs.Enqueue(job);
        std::cout << "\nSuccessfully added job: ";
        job.display();
        std::cout << std::endl;
    }

    //processess the next job
    void processNextJob() {
        if (pendingJobs.empty()) {
            std::cout << "\nNo pending jobs to process." << std::endl;
            return;
        }

        Job currentJob = pendingJobs.Front();
        std::cout << "\nProcessing... ";
        currentJob.display();
        std::cout << std::endl;

        pendingJobs.Dequeue();
        std::cout << "Finished Job ID: " << currentJob.id << std::endl;
    }

```

```
128
129     void processAllJobs() {
130         if (pendingJobs.empty()) {
131             std::cout << "\nNo pending jobs in the queue." << std::endl;
132             return;
133         }
134
135         //loop that process all the jobs
136         std::cout << "\n--- Processing All Pending Jobs ---" << std::endl;
137         while (!pendingJobs.empty()) {
138             processNextJob();
139         }
140         std::cout << "--- All print jobs completed ---" << std::endl;
141     }
142 };
143
144 #endif // PRINTER_SYSTEM_H
```

MAIN

```cpp
#include <iostream>
#include <string>
#include "PrinterSystem.h"

// Helper function to safely read integers without extra library headers
int getValidInt(const std::string& prompt) {
    int value;
    while (true) {
        std::cout << prompt;
        if (std::cin >> value) {
            return value;
        } else {
            std::cout << "Invalid input! Please enter a numeric value.\n";
            std::cin.clear(); // Reset cin error flag
            while (std::cin.get() != '\n'); // Loop to read and discard bad characters until newline
        }
    }
}

int main() {
    int capInput = getValidInt("Enter Printer Queue Capacity: ");
    while (capInput <= 0) {
        std::cout << "Capacity must be greater than 0.\n";
        capInput = getValidInt("Enter Printer Queue Capacity: ");
    }

    Printer officePrinter(static_cast<size_t>(capInput));

    int choice = 0;
    do {
        std::cout << "\n========== PRINTER SYSTEM MENU ==========\n";
        std::cout << "1. Add Print Job\n";
        std::cout << "2. Process Next Job\n";
        std::cout << "3. Process All Jobs\n";
        std::cout << "4. View Queue Status\n";
        std::cout << "5. Exit\n";

        choice = getValidInt("Enter choice (1-5): ");

        switch (choice) {
            case 1: {
                int id = getValidInt("\nEnter Job ID: ");

                std::string userName;
                std::cout << "Enter User Name: ";
                while (std::cin.get() != '\n'); // Clear trailing newline left by integer input
                std::getline(std::cin, userName);

```

```
        switch (choice) {
            case 1: {
                int id = getValidInt("\nEnter Job ID: ");

                std::string userName;
                std::cout << "Enter User Name: ";
                while (std::cin.get() != '\n'); // Clear trailing newline left by integer input
                std::getline(std::cin, userName);

                int pages = getValidInt("Enter Number of Pages: ");

                officePrinter.addJob(Job(id, userName, pages));
                break;
            }
            case 2:
                officePrinter.processNextJob();
                break;
            case 3:
                officePrinter.processAllJobs();
                break;
            case 4:
                std::cout << "\nQueue Status: " << officePrinter.getPendingCount()
                          << " / " << officePrinter.getCapacity() << " jobs pending.\n";
                break;
            case 5:
                std::cout << "\nExiting Printer Simulation.\n";
                break;
            default:
                std::cout << "\nInvalid choice! Please enter a number from 1 to 5.\n";
        }
    } while (choice != 5);

    return 0;
}
```

OUTPUT

```
Queue created with capacity: 6
q_front: 1, q_back: 1, q_size: 1
q_front: 1, q_back: 2, q_size: 2
q_front: 1, q_back: 3, q_size: 3
q_front: 1, q_back: 4, q_size: 4
q_front: 1, q_back: 5, q_size: 5
q_front: 1, q_back: 6, q_size: 6
q_front: 2, q_back: 6, q_size: 5
q_front: 3, q_back: 6, q_size: 4
q_front: 4, q_back: 6, q_size: 3
q_front: 5, q_back: 6, q_size: 2
q_front: 6, q_back: 6, q_size: 1
q_front: Queue is empty.
```

```
========== PRINTER SYSTEM MENU ==========
1. Add Print Job
2. Process Next Job
3. Process All Jobs
4. View Queue Status
5. Exit
Enter choice (1-5): 1

Enter Job ID: 1101
Enter User Name: ram
Enter Number of Pages: 3

Successfully added job: [Job ID: 1101 | User: ram | Pages: 3]

========== PRINTER SYSTEM MENU ==========
1. Add Print Job
2. Process Next Job
3. Process All Jobs
4. View Queue Status
5. Exit
Enter choice (1-5): 1

Enter Job ID: 1102
Enter User Name: gian
Enter Number of Pages: 4

Successfully added job: [Job ID: 1102 | User: gian | Pages: 4]

========== PRINTER SYSTEM MENU ==========
1. Add Print Job
2. Process Next Job
3. Process All Jobs
4. View Queue Status
5. Exit
Enter choice (1-5): 1

Enter Job ID: 1103
Enter User Name: dreu
Enter Number of Pages: 1

Successfully added job: [Job ID: 1103 | User: dreu | Pages: 1]

========== PRINTER SYSTEM MENU ==========
1. Add Print Job
2. Process Next Job
3. Process All Jobs
4. View Queue Status
5. Exit
Enter choice (1-5): 1

Enter Job ID: 1104
Enter User Name: ryan
Enter Number of Pages: 3
```

```
Enter User Name: ryan
Enter Number of Pages: 3

Error: Printer queue is full! Unable to add [Job ID: 1104 | User: ryan | Pages: 3]

========== PRINTER SYSTEM MENU ==========
1. Add Print Job
2. Process Next Job
3. Process All Jobs
4. View Queue Status
5. Exit
Enter choice (1-5): 4

Queue Status: 3 / 3 jobs pending.

========== PRINTER SYSTEM MENU ==========
1. Add Print Job
2. Process Next Job
3. Process All Jobs
4. View Queue Status
5. Exit
Enter choice (1-5): 2

Processing... [Job ID: 1101 | User: ram | Pages: 3]
Finished Job ID: 1101

========== PRINTER SYSTEM MENU ==========
1. Add Print Job
2. Process Next Job
3. Process All Jobs
4. View Queue Status
5. Exit
Enter choice (1-5): 4

Queue Status: 2 / 3 jobs pending.

========== PRINTER SYSTEM MENU ==========
1. Add Print Job
2. Process Next Job
3. Process All Jobs
4. View Queue Status
5. Exit
Enter choice (1-5): 3

--- Processing All Pending Jobs ---

Processing... [Job ID: 1102 | User: gian | Pages: 4]
Finished Job ID: 1102

Processing... [Job ID: 1103 | User: dreu | Pages: 1]
Finished Job ID: 1103
--- All print jobs completed ---
```

```
========== PRINTER SYSTEM MENU ==========
1. Add Print Job
2. Process Next Job
3. Process All Jobs
4. View Queue Status
5. Exit
Enter choice (1-5): 4

Queue Status: 0 / 3 jobs pending.

========== PRINTER SYSTEM MENU ==========
1. Add Print Job
2. Process Next Job
3. Process All Jobs
4. View Queue Status
5. Exit
Enter choice (1-5): 5

Exiting Printer Simulation.
PS C:\Users\PC\Documents\DSA CAMPO\HOA5p1-20260811T052416Z-1-001\ILO C>
```

Summary of lessons learned

I learned how to create a queue and structure the logic and operations(enqueueing, dequeueing, display all contents, isFull and is Empty deletion, etc) of the queue in both array and linked lists, again both serving its own purposes since arrays are easier to access data immediately while linked lists doesn't have a fixed sizes. I also learned how queue works acting as a line of accessing data applying the saying “first come, first serve”

Analysis of the procedure

For ILO A and ILO B its just a simple implementation of the queue in both linked list and array and how their operations to structure the queue’s logic works. ILO A contains the constructor enqueuing, dequeueing, display all values and return the first value for ILOB it also contains enqueueing and dequeueing while adding and the operators from ILOA but this time it handles the back of the queue aswell

Analysis of the supplementary activity

The supplementary has 3 classes one for the array, one for the job itself and one for the printing of the data. The reason why I chose an array approach instead of linked list because in a real setting you have a limit of a how many job applications you can take, plus for me its easier to access the data directly in the array approach rather than the linked list where u have to traverse through the data, wherein array you can just access the data immediately. This program also uses a main menu so that the user can see the job applications and process them

Concluding statement / Feedback: How well did you think you did in this activity? What are your areas

for Improvement?

For me I think so far im getting used to coding linked lists and arrays but far from being satisfactory with my skills, I would also like to point out that there is no code provided anymore which is an increase of difficulty for me

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.